

CS 6220 — Data Mining Techniques

Homework 1: Report

Shashank Jajimoggala

NUID: 002570569

September 23, 2026

Repository: <https://github.com/shashankJ52/CS6220-Data-Mining>

Q1. Did any student in class help you in a major way with the setup process?

No student assisted with the setup process.

Q2. Show the link to your repository with source code on GitHub.

<https://github.com/shashankJ52/CS6220-Data-Mining>

The notebook `homework_1.ipynb` is organised into numbered sections, each introduced by a markdown cell explaining the purpose of the step and the reasoning behind the choices made. Code cells carry comments on the non-obvious parts, and helper functions are placed in separate cells from the cells that call them. All cells were run in order before the final commit, so outputs are visible in the committed file.

The repository root contains `README.md`, describing the steps to clone, install dependencies, register the Jupyter kernel and run the notebook, and `requirements.txt`, listing the packages required. Data files are not tracked in the repository; the README documents where to obtain them.

Q3. Briefly explain your environment.

a. What programming language are you using?

Python 3.11.15.

b. What tool do you use to write and debug the source code?

Visual Studio Code, using its built-in notebook editor (provided by the Jupyter extension). Debugging is done through VS Code's variable inspector and by executing cells individually to check intermediate results.

c. What tool do you use to test and execute the code?

Code executes in a Jupyter kernel — specifically Python (CS6220), an ipykernel registration pointing at the project's virtual environment. The kernel is driven either from VS Code's notebook interface or from JupyterLab in the browser; both connect to the same kernel, so results are identical in either.

d. Are you using scikit-learn?

Yes, scikit-learn 1.9.1. Also used: pandas 3.0.6 for data handling, and matplotlib 3.11.2 with seaborn 0.13.2 for the scatterplot matrix.

e. Are you running your program on your laptop?

Yes — locally on an Apple Silicon MacBook, with no cloud service. Dependencies are isolated in a Python virtual environment (venv), registered as a named Jupyter kernel (Python (CS6220)) so that both JupyterLab and VS Code execute against the same interpreter.

Q4. Show a screenshot of your development environment with relevant code.

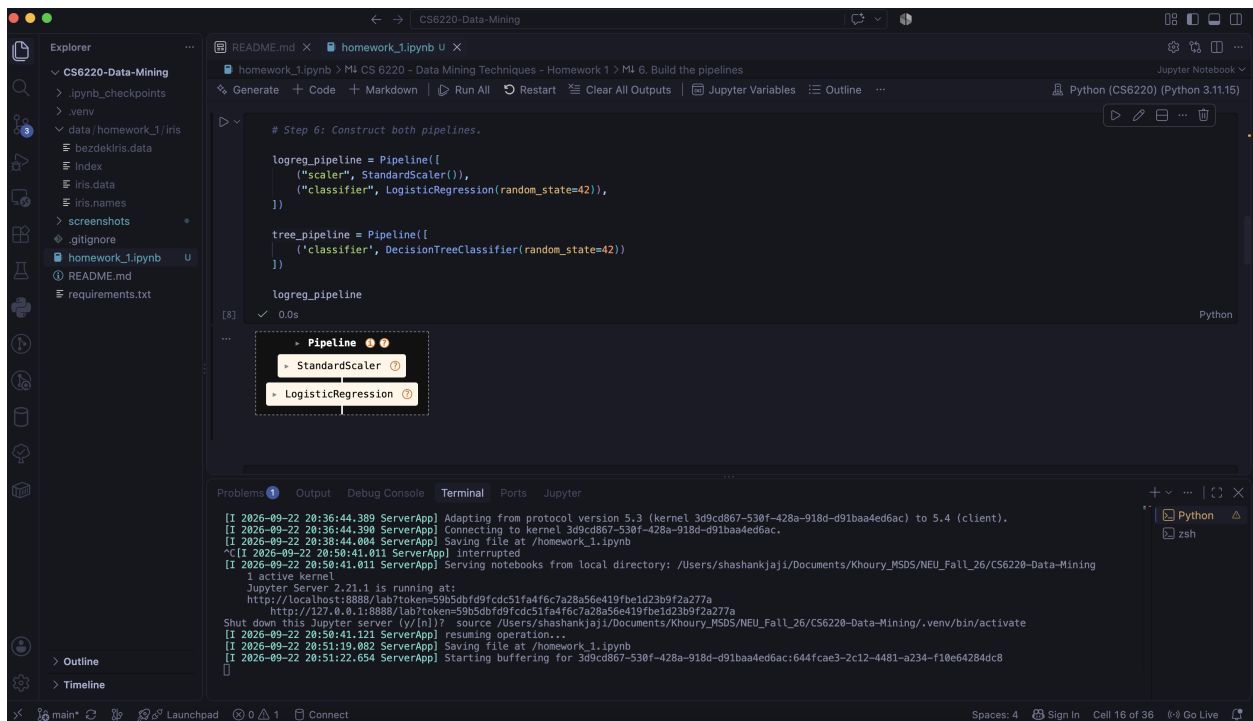


Figure 1. VS Code with `homework_1.ipynb` open, showing the cell that constructs both pipelines. The logistic regression pipeline includes a `StandardScaler`; the decision tree pipeline does not. The repository structure is visible in the explorer, and the active kernel Python (CS6220) (Python 3.11.15) appears at the top right.

Q5. Show a screenshot of your program execution, including some intermediate and/or final results.

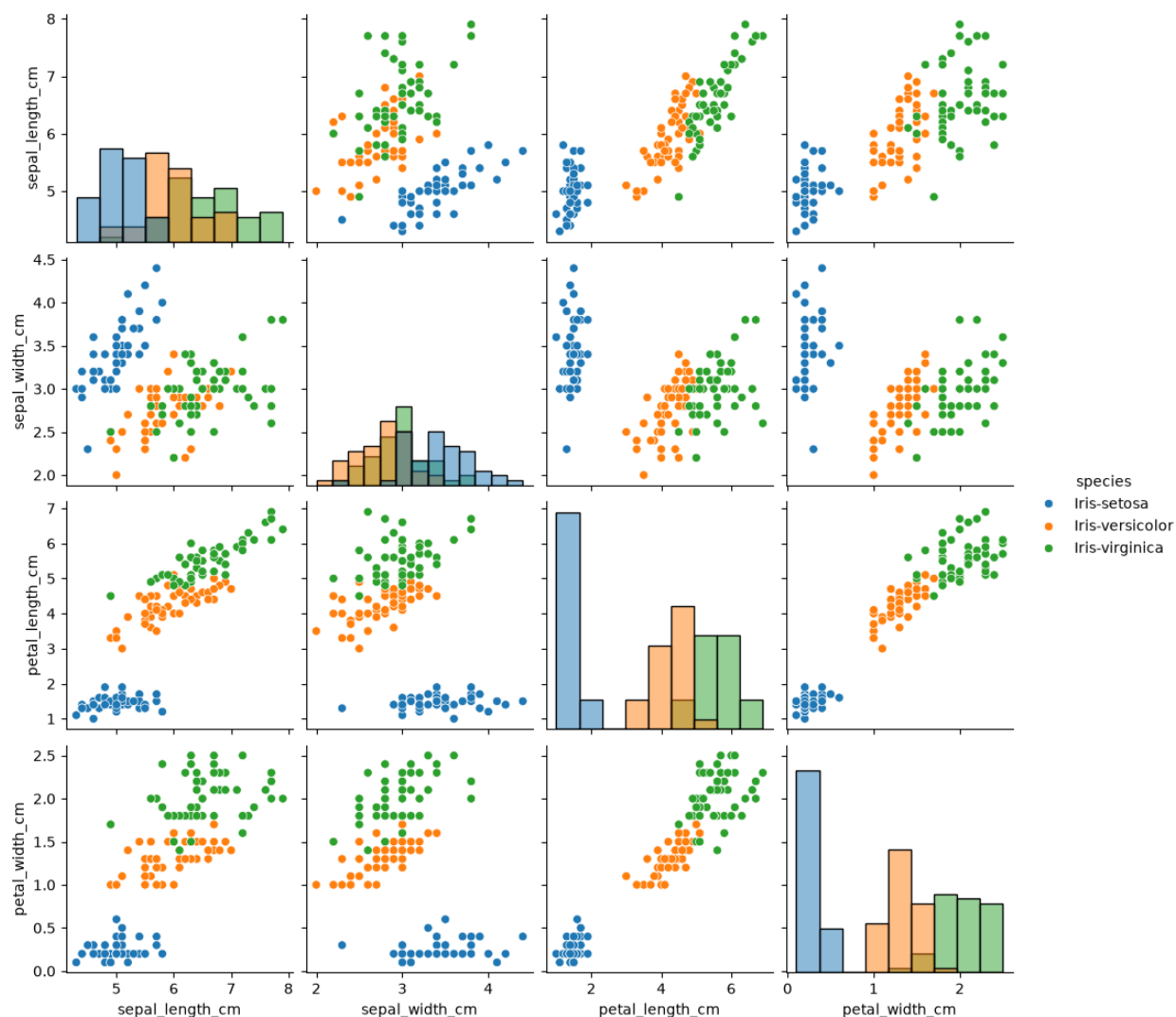


Figure 2. Scatterplot matrix of all four measurements, coloured by species, produced by the notebook. Setosa (blue) forms a cluster separated from the other two species in every panel involving a petal measurement. Versicolor (orange) and virginica (green) are adjacent, almost separable on petal length and petal width but heavily overlapping on the sepal measurements. This visualisation informed the prediction, made before any model was fitted, that setosa would be classified perfectly and that all errors would fall between the other two species.

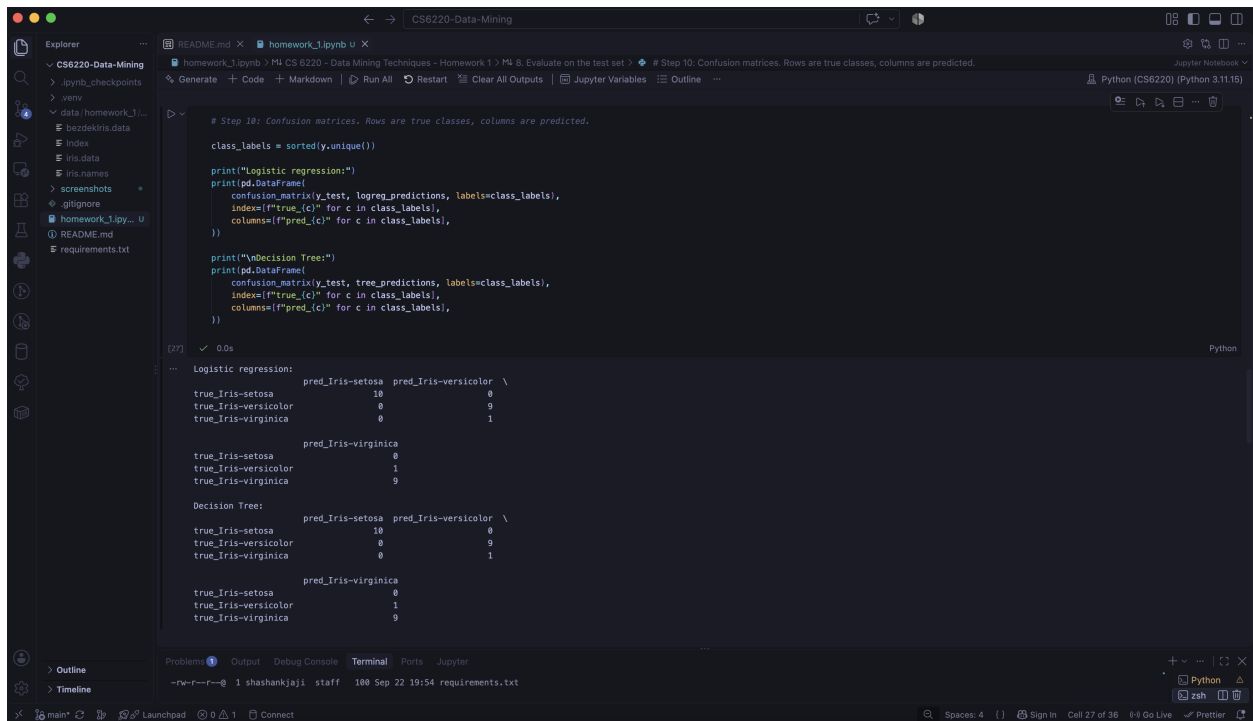


Figure 3. Confusion matrices for both models on the 30 held-out test flowers. Rows are true classes, columns are predicted. Both models classified setosa perfectly and made one error in each direction between versicolor and virginica.

Q6. Run your pipeline on the Iris flower data.

Two pipelines were built and compared: logistic regression with feature standardisation, and a decision tree without. The scaler is included for logistic regression because that model combines all four features into a single weighted sum, so features with wider numeric ranges carry more influence — petal length has a standard deviation of 1.76 cm against 0.43 cm for sepal width, a ratio of roughly 4:1. The decision tree examines one feature at a time against a threshold, and standardisation preserves the ordering of values within a feature, so it cannot change which splits the tree selects.

The data was split 80/20 with `train_test_split`, stratified by species so that the 30-row test set contains exactly 10 of each class.

Timings were measured with `time.perf_counter`, chosen over `time.time` because it is monotonic and higher-resolution. Both models complete in single-digit milliseconds on this dataset, and single measurements proved unreliable: an initial attempt averaging 10 runs indicated the decision tree predicted three times slower than logistic regression, contradicting the mechanics of the two algorithms. Increasing to 100 runs reversed the ordering and matched expectation, identifying the earlier figure as measurement noise. All figures below are means over 100 runs, excluding the first, which carries one-time library initialisation costs.

a. Running time for training

Model	First run	Mean of warm runs
Logistic regression	7.955 ms	2.714 ms
Decision tree	1.352 ms	0.943 ms

b. Running time for testing (prediction on 30 held-out rows)

Model	First run	Mean of warm runs
Logistic regression	4.166 ms	0.698 ms
Decision tree	0.556 ms	0.402 ms

The decision tree is faster at both stages — roughly $2.9\times$ at training and $1.7\times$ at prediction — consistent with logistic regression running an iterative numerical solver while the tree performs direct threshold comparisons.

c. Prediction accuracy on the test data

Both models achieved **0.9333** (28 of 30 correct).

The two models produced identical confusion matrices and, compared row by row, made the same prediction for every test flower. All errors fell between versicolor and virginica; setosa was classified perfectly. This matches the prediction made from the scatterplot matrix before any model was trained: setosa forms a cluster separated from the other two species, while versicolor and virginica sit adjacent with a few points near the boundary.

With only 30 test rows, a single misclassification shifts accuracy by 3.33 percentage points, so this estimate is coarse and specific to this split.

Hyperparameter exploration (optional task)

Each candidate value was scored by 5-fold cross-validation on the training set only, so that the test set influenced no modelling decision.

For logistic regression, cross-validated accuracy rose from 0.833 at $C = 0.001$ to a plateau of 0.967 at $C = 10$ and above. The differences among $C = 1, 10$ and 100 are smaller than the standard deviation across folds (≈ 0.03) and are not meaningful; the default $C = 1.0$ is a reasonable choice.

For the decision tree, `max_depth = 1` scored exactly 0.6667 with zero variance across folds. This follows from the structure: one split produces two leaves, so one of the three classes cannot be predicted at all, and the failure is identical in every fold. Depth 2 makes all three classes reachable and accuracy jumps to 0.925. Performance plateaus at depth 4 (0.9417), with depths 5, 6 and unrestricted returning identical scores — the additional levels fit fold-specific noise without improving generalisation.

max_depth	Mean CV accuracy	Std. dev.
1	0.6667	0.0000
2	0.9250	0.0312
3	0.9333	0.0204
4	0.9417	0.0204
5	0.9417	0.0204
6	0.9417	0.0204
None	0.9417	0.0204

Conclusion

Accuracy cannot distinguish these two models on this dataset. The choice would rest on other criteria: the decision tree is faster at both training and prediction, requires no scaling step, and can be read directly as a sequence of threshold comparisons. Logistic regression provides probability estimates and coefficients indicating how each measurement influences the prediction.

AI tool usage

Per the syllabus, AI assistance was used in producing this assignment. Claude was used as a tutor: concepts were explained and my reasoning checked before code was written, rather than code being generated wholesale. The prompts issued, in order, were:

1. I need to make a github repo for my CS6220 class as mentioned. [course handout pasted] Guide me on how to do it.
2. When I run the same thing on my VS Code in the same file it asks me to select a Python kernel. Which one to choose?
3. You are my Coding Buddy/Mentor. I will give you my assignment and also my understanding of it. Then you can correct me on how to go about it, where I went wrong. The goal is not to do the entire homework for me at once, but to help me solve the homework by myself step by step.
4. My understanding: [my own reading of the tasks, pasted].
5. Give me an overview of all the classifiers I can choose from so I have an idea of what they are in general and their respective use cases.
6. Let's do it in two phases, one time with logistic regression and one time with decision tree, and at the end of those phases evaluate the models separately and then compare both models. How does that sound?
7. I'm not sure about the StandardScaler thing. Can you explain what that is, and how the decision is to be thought about?
8. My answer on why logistic regression needs scaling and the decision tree does not.
9. Do I need to import the dataset from `sklearn.datasets import load_iris`, or download it from the UCI repo?
10. (My observations from the scatterplot matrix.)
11. Is this right? [my `train_test_split` call, missing `stratify`].
12. Is this right? [my incorrect `time_fit` call].
13. Give me the code for prediction timing on the test set, for syntax purposes. I'll write the next one.
14. Give me the code block for the hyperparameter sweep so I can see and learn this time.